

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра радіотехнічних систем

ЗВІТ

до лабораторної роботи №7

з дисципліни «Програмовані засоби в інтелектуальній радіоелектронній техніці»
«Реалізація багатозадачності у вбудованих системах»

Студент: Кравченко Артем

Група: РЕ-41

Викладач: _____

Київ 2026

Мета роботи

Ознайомитись із методами реалізації багатозадачності у вбудованих системах на базі мікроконтролера PIC18F4550, а також отримати практичні навички роботи з таймерами, перериваннями, пріоритетами переривань і розподілом процесорного часу між кількома задачами.

Короткий план виконання роботи

1. Створено проєкт для мікроконтролера PIC18F4550 у середовищі MPLAB C18 з урахуванням роботи через bootloader.
2. Реалізовано шаблон з переадресацією векторів переривань та налаштовано пріоритетні переривання.
3. Ініціалізовано периферійні модулі TMR0, USART і CCP1.
4. У спільному проєкті реалізовано навчальні приклади з багатозадачності.
5. Перевірено роботу обробників переривань, реакцію на натискання кнопок та формування ШІМ-сигналу.
6. Виконано тестування проєкту в симуляторі.

Код програми

```
1 #include <pi18f4550.h>
2 #include <delays.h>
3
4 #define K1 0x01
5 #define K2 0x02
6 #define K3 0x04
7 #define PIN_Keys PORTE
8 #define PIN_K1 PORTE
9 #define PIN_K2 PORTE
10 #define PIN_K3 PORTE
11 #define Keypad (K1 | K2 | K3)
12
13
14 unsigned int counter;
15 unsigned int keycptr;
16 struct flagsStruct
17 {
18     unsigned int clicking;
19     unsigned int pushing;
20 };
21 struct flagsStruct flags;
22 unsigned char press_code;
23 unsigned int adc_value;
24 char buffer[6];
25 unsigned char k;
26
27 void systemInit(void);
28 void TMR0Init(void);
29 void USARTInit(void);
30 void ADCInit(void);
31 void PWMInit(void);
32
33 void PWMInit(void);
34
35 void TMR0Interrupt(void);
36
37 void sendCharUSART(char);
38 void sendWordUSART(char*);
39 void numberToWord(unsigned int, char*);
40
41 unsigned char keyboard(void);
42 void PWMSetup(unsigned char);
43
44 extern void _startup (void);
45 void ISRHigh(void);
46
47 #pragma code main=0x102A
48 void main(void)
49 {
50     TRISB = 0;
51     ADCON1 = 0x0F;
52     counter = 1;
53     keycptr = 0;
54     adc_value = 0;
55
56     systemInit();
57     TMR0Init();
58     USARTInit();
59     ADCInit();
60     PWMInit();
61
62     while (1)
63     {
64         sendCharUSART('\f');
65         if (press_code & 128) sendCharUSART('L');
66         if (press_code)
67         {
68             unsigned char multiplier;
69
70             sendCharUSART('B');
71             sendCharUSART((press_code & 7) | '0');
72             multiplier = 1 << ((press_code & 7) - 1);
73             multiplier *= (press_code & 128) ? (8) : 1;
74             sendCharUSART('M');
75             numberToWord(multiplier, buffer);
76             sendWordUSART(buffer);
77             PWMSetup(multiplier);
78         }
79
80         for (k = 0; k < 1; k++)
81         {
82             Delay10KTCYx(250);
83         }
84     }
85
86 #pragma code interrupt_high_vector=0x1008
87 void interrupt_high_vector(void)
88 {
89     __asm goto ISRHigh __endasm
90 }
91
92 #pragma code
93
94 #pragma interrupt ISRHigh
95 void ISRHigh(void)
96 {
97     if (INTCONbits.TMR0IF)
98     {
99         INTCONbits.TMR0IF = 0;
100         TMR0Interrupt();
101     }
102     // Delay10KTCYx(250);
103 }
104
105 void systemInit(void)
106 {
107     INTCONbits.GIE = 1;
108     INTCONbits.PEIE = 1;
109 }
110
111 void TMR0Init(void)
112 {
113     TOCON = 0b10010001;
114     INTCONbits.TMR0IE = 1;
115     INTCON2bits.TMR0IF = 1;
116 }
117
118 void USARTInit(void)
119 {
120     SPBRGH = 0;
```

```

120 {
121     SPBRGH = 0;
122     BAUDCONbits.BRG16 = 0;
123
124     SPBRG = 77;
125     TXSTAbits.BRGH = 0;
126     TXSTAbits.SYNC = 0;
127     RCSTAbits.SPEN = 1;
128
129     TXSTAbits.TX9 = 0;
130     TXSTAbits.TXEN = 1;
131 }
132
133 void ADCInit(void)
134 {
135     ADCON0 = 0b00000001;
136     ADCON1 = 0b00001110;
137     ADCON2 = 0b00101110;
138
139     ADCON0bits.GO = 1;
140 }
141
142 void PWMInit(void)
143 {
144     CCP1CON = 0b00001100;
145     T2CON = 0b00000000;
146     TRISCbits.RC1 = 0;
147     TRISCbits.RC2 = 0;
148
149     TRISBbits.RB3 = 0;
150 }
151
152 void TMR0Interrupt(void)
153 {
154     unsigned char current_press_code = keyboard();
155     if (current_press_code) press_code = current_press_code;
156 }
157
158 void PWMSetup(unsigned char multiplier)
159 {
160     if (multiplier)
161     {
162         T2CONbits.TMR2ON = 1;
163         if (multiplier != 32)
164         {
165             T2CONbits.T2CKPS = 2;
166             PR2 = (240 / multiplier) - 1;
167         }
168         else if (multiplier == 32)
169         {
170             T2CONbits.T2CKPS = 1;
171             PR2 = 29;
172         }
173         CCP1L = (PR2 + 1) / 2;
174     }
175     else
176     {
177         T2CONbits.TMR2ON = 0;
178     }
179 }
180
181 void sendCharUSART(char data)

```

```

180
181 void sendCharUSART(char data)
182 {
183     while(TXSTAbits.TXMT == 0);
184     TXREG = data;
185 }
186
187 void sendWordUSART(char* word)
188 {
189     int i = 0;
190     // sendCharUSART('\f');
191     while(word[i] != '\0' && i != 5)
192     {
193         sendCharUSART(word[i]);
194         i++;
195     }
196     // sendCharUSART('\r');
197 }
198
199 void numberToWord(unsigned int number, char* word)
200 {
201     register unsigned char n;
202     for(n = 0; n <= 4; n++)
203     {
204         word[n] = '0';
205     }
206     for(n = 0; n <= 4; n++)
207     {
208         word[4-n] = (number % 10) | '0';
209         number /= 10;
210         if (!number) break;
211     }

```

```

210         if (!number) break;
211     }
212 }
213
214 unsigned char keyboard(void)
215 {
216     struct flags
217     {
218         unsigned int clicking;
219         unsigned int pushing;
220     };
221     unsigned char instant = (PIN_Keys & Keypad) & 0b00000111;
222
223     if (instant != Keypad)
224     {
225         int i;
226         unsigned char button_counter = 0;
227         for (i = R1; i <= R3; i+=2)
228         {
229             button_counter++;
230             if (!(instant & i))
231             {
232                 if (!flags.clicking)
233                 {
234                     flags.clicking = 1;
235                     return button_counter;
236                 }
237             }
238             if ((!(flags.pushing) && (keycnt > 12))
239             {
240                 flags.pushing = 1;

```

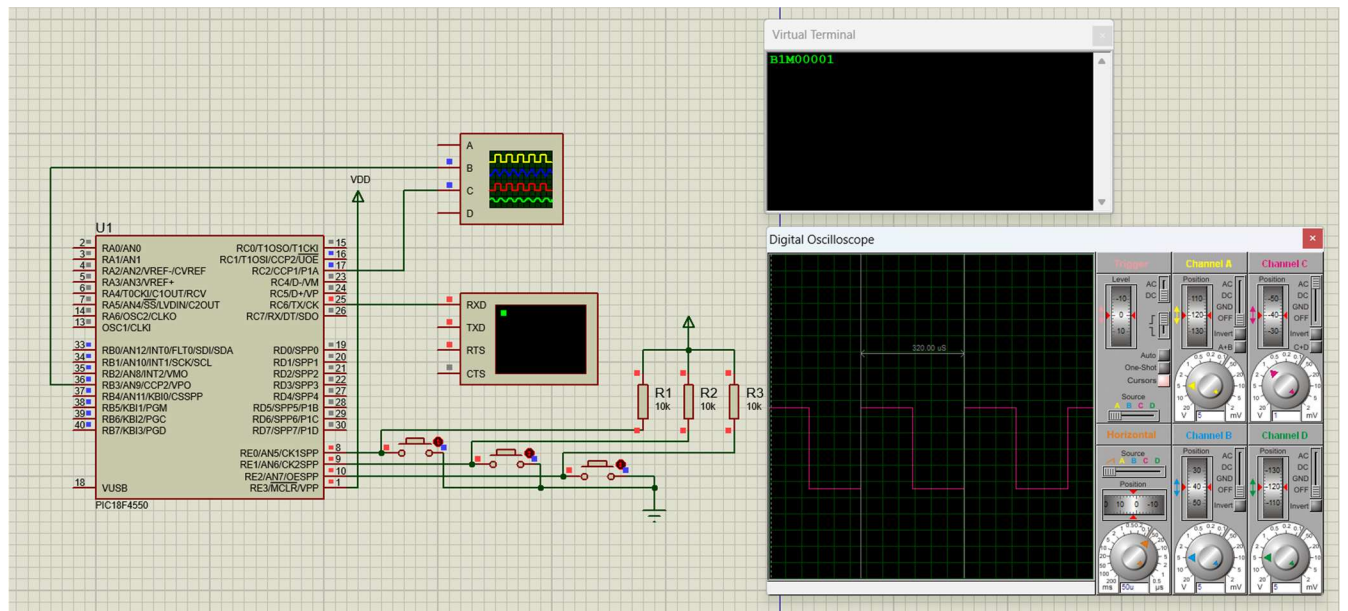
```

240         flags.pushing = 1;
241         return button_counter | 0x80;
242     }
243     keycntr++;
244 }
245 }
246 }
247 else
248 {
249     flags.pushing = 0;
250     flags.clicking = 0;
251     keycntr = 0;
252 }
253 return 0;
254 }

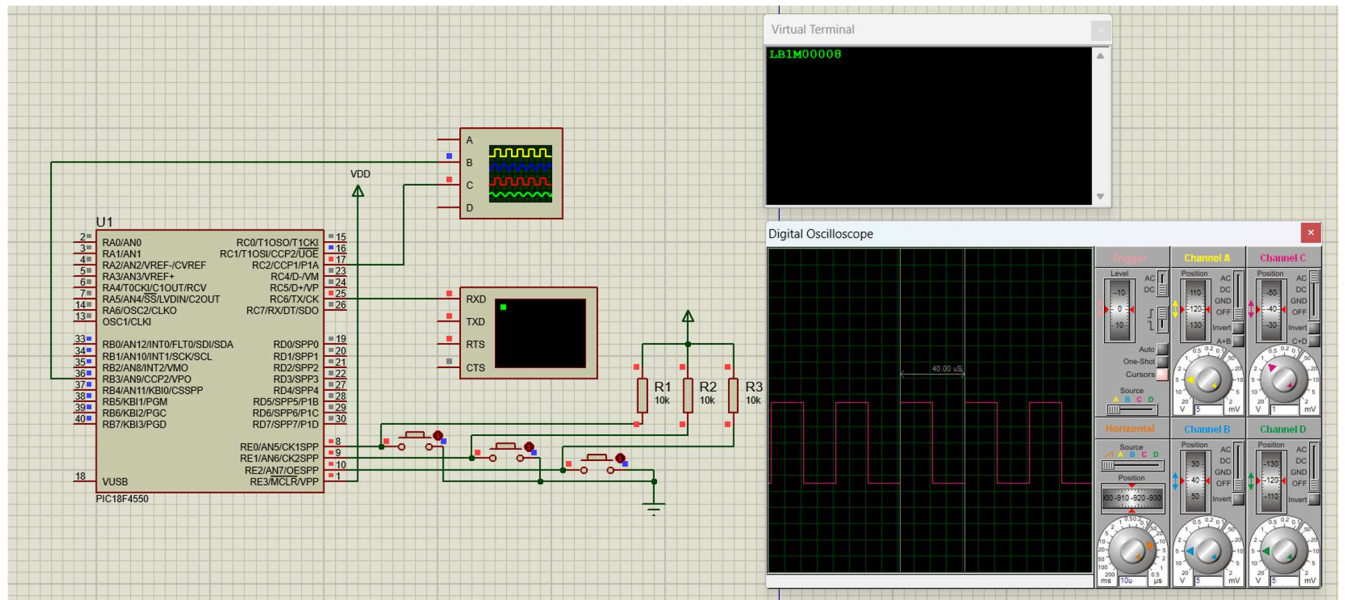
```

Результат виконання у середовищі Proteus (детальніше у демонстрації):

Звичайне натиснення кнопки 1:



Тривали натиснення кнопки 1:



Вивід у UART в такому форматі: «Тип натиснення (В – коротке, LB - тривале)» + «Номер кнопки» + «Множник частоти»

На осцилографі можна бачити період утвореного ШІМ сигналу (меандру).

Контрольні запитання

1. Поясніть, що таке дребезг контакту і чому він може бути особливо критичним при використанні зовнішніх переривань (INTx).

Дребезг контакту — це багаторазове коротке замикання і розмикання контактів кнопки в момент натискання або відпускання. Для зовнішніх переривань це критично, бо один фізичний натиск може бути сприйнятий як кілька окремих подій, через що програма виконає обробник переривання декілька разів.

2. Як впливає на програму наявність програмного завантажувача і навіщо потрібна переадресація векторів переривань?

Bootloader займає початкову область пам'яті програм, тому основний код і стандартні вектори переривань не можуть розміщуватись у звичних адресах. Через це застосовується переадресація: переривання спочатку потрапляє у вектор bootloader, а далі передається на адресу обробника користувацької програми.

3. Поясніть, що таке перемикавання контексту та як апаратна частина МК PIC18F4550 забезпечує швидке збереження контексту для високопріоритетних переривань?

Перемикавання контексту — це тимчасове зупинення основної програми або іншої задачі та перехід до обробника переривання з подальшим поверненням назад. У PIC18F4550 для швидкого обслуговування високопріоритетних переривань апаратно зберігаються адреса повернення, а також можуть використовуватись тіньові регістри для WREG, STATUS і BSR.

4. Яке призначення мають біти GIEH та GIEL у регістрі INTCON у режимі пріоритетних переривань?

Біт GIEH дозволяє або забороняє всі високопріоритетні переривання, а біт GIEL — усі низькопріоритетні. У режимі пріоритетів ці біти дають змогу окремо керувати двома рівнями переривань.

5. У чому полягає відмінність між інструкціями RETFIE та RETFIE FAST?

RETFIE виконує звичайне повернення з переривання з відновленням виконання програми. RETFIE FAST додатково використовує швидке відновлення з тіньових регістрів, тому застосовується переважно для високопріоритетних переривань і працює швидше.

6. Як реалізується багатозадачність (симуляція паралельного виконання) за допомогою двох таймерів?

Кожен таймер періодично генерує своє переривання, а у відповідній ISR виконується окрема задача або службова дія. Якщо таймери мають різні періоди та пріоритети, мікроконтролер послідовно виконує кілька задач у різні моменти часу, що створює ефект паралельної роботи.

7. Якщо під час обслуговування TMR1 (низький пріоритет) спрацюває переривання RCIF, яке має високий пріоритет, яка послідовність дій МК?

Мікроконтролер тимчасово перериває обробку низькопріоритетного ISR від TMR1, переходить до високопріоритетного обробника RCIF, виконує його, а після завершення повертається назад до перерваного обробника TMR1. Саме так забезпечується перевага важливіших подій.

Висновок

У ході виконання лабораторної роботи було досліджено принципи реалізації багатозадачності у вбудованих системах на базі PIC18F4550. У спільному проєкті було поєднано роботу таймерів, переривань, PWM та оброблення кнопок різної тривалості натискання. У результаті підтверджено, що навіть без використання RTOS можна організувати кілька незалежних задач за допомогою часових слотів і пріоритетних переривань. Отримані навички можуть бути використані при побудові складніших вбудованих систем реального часу.